

CLIENT DELIVERABLE

Utopia Deli — No-Show Prevention System

Systack Documentation

ROLE	COLOR	HEX
Primary	Deep Burgundy	 #590B3F
Primary Light	Burgundy Light	 #7a1a55
Accent	Rust Red	 #AF3D4B
Accent Hover	Rust Light	 #c44d5b
Secondary	Purple	 #754681
Gold	Warm Gold	 #D59F5C
Gold Light	Cream	 #f5e6d0
Background	Off-White	 #FBFCFE
Card	White	 #FFFFFF
Text	Dark Gray	 #1F2937
Text Light	Medium Gray	 #6B7280
Border	Light Gray	 #E5E7EB
Success	Green	 #22c55e
Error	Red	 #dc2626

Utopia Deli — No-Show Prevention System

Workflow Walkthrough

Document ID: UD-NSHOW-WF-001

Version: 1.0

Status: Internal — Systack Only

Source System: Utopia Deli (Partial — Phase 1 complete)

Date: 2026-06-16

Builder: SOL / Assembly

Brand Reference

ROLE	COLOR	HEX
Primary	Deep Burgundy	#590B3F
Primary Light	Burgundy Light	#7a1a55

ROLE	COLOR	HEX
Accent	Rust Red	#AF3D4B
Accent Hover	Rust Light	#c44d5b
Secondary	Purple	#754681
Gold	Warm Gold	#D59F5C
Gold Light	Cream	#f5e6d0
Background	Off-White	#FBFCFE
Card	White	#FFFFFF
Text	Dark Gray	#1F2937
Text Light	Medium Gray	#6B7280
Border	Light Gray	#E5E7EB
Success	Green	#22c55e
Error	Red	#dc2626

Workflow 1: Create Booking

Trigger: POST /systack-create-booking

Node 1 — Webhook

Receives booking data:

- Customer name, email, phone
- Service type
- Slot datetime
- Total value

Node 2 — Calculate Deposit

Type: Code Node

Applies deposit rules:

```
const totalValue = $json.total_value;
let depositPercent = 25;
let minDeposit = 1000; // $10.00

if (totalValue < 5000) depositPercent = 100;    // < $50
else if (totalValue < 15000) depositPercent = 50; // $50-$150

let depositAmount = Math.round(totalValue * depositPercent / 100);
if (depositAmount < minDeposit) depositAmount = minDeposit;
```

```
return [{ json: { ...$json, deposit_amount: depositAmount } }];
```

Node 3 — Generate Confirmation Token

Type: Code Node

Generates unique cryptographic token for confirm/cancel links.

Node 4 — Collect Deposit via Square

Type: HTTP Request (Square API)

Creates payment link for deposit amount.

Node 5 — Insert into Database

Type: Postgres Node

```
INSERT INTO bookings (  
  booking_id, customer_name, customer_email, customer_phone,  
  service_type, slot_datetime, deposit_amount, total_value,  
  status, confirmation_token  
) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, 'pending', $9)
```

Node 6 — Send Confirmation Email

Type: Email Send (SMTP)

Sends to customer:

- Booking details
 - Deposit confirmation
 - Confirm/Cancel buttons with token links
 - Cancellation policy summary
-

Workflow 2: Confirm Handler

Trigger: GET /systack-confirm-booking?token=xxx

Node 1 — Webhook

Receives token from email link click.

Node 2 — Validate Token

Type: Code Node

Looks up booking by token. Checks:

- Token exists in database
 - Booking not already confirmed/cancelled
 - Booking not past its slot time
-

Node 3 — Update Status

Type: Postgres Node

```
UPDATE bookings
SET status = 'confirmed', confirmed_at = NOW()
WHERE confirmation_token = $1 AND status = 'pending'
```

Node 4 — Respond

Returns confirmation page to customer:

- “Booking Confirmed” message
 - “You’ll receive reminders 24h and 2h before”
-

Workflow 3: T-24h Reminder

Trigger: Schedule (Cron)

Node 1 — Find Bookings Due

Type: Postgres Node

```
SELECT * FROM bookings
WHERE slot_datetime BETWEEN NOW() + INTERVAL '23 hours'
AND NOW() + INTERVAL '25 hours'
AND status = 'confirmed'
AND reminder_24h_sent = FALSE
```

Node 2 — Send Reminder Email

Type: Email Send (SMTP)

Content:

- “Your booking is tomorrow!”
 - Confirm/Cancel buttons with token
 - Cancellation policy: “Cancel > 24h ahead for full deposit refund”
-

Node 3 — Mark Reminder Sent

Type: Postgres Node

```
UPDATE bookings SET reminder_24h_sent = TRUE WHERE booking_id = $1
```

Workflow 4: T-2h Reminder

Trigger: Schedule (Cron)

Status: Built, needs testing

Node 1 — Find Bookings Due

```
SELECT * FROM bookings
WHERE slot_datetime BETWEEN NOW() + INTERVAL '1 hour 55 minutes'
    AND NOW() + INTERVAL '2 hours 5 minutes'
    AND status = 'confirmed'
    AND reminder_2h_sent = FALSE
```

Node 2 — Send Urgent Reminder

Content:

- “Your booking is in 2 hours!”
- “I’m on my way” confirmation button
- “Unconfirmed slots may be released”

Node 3 — Mark Reminder Sent

```
UPDATE bookings SET reminder_2h_sent = TRUE WHERE booking_id = $1
```

Workflow 5: Auto-Release

Trigger: Schedule (Cron)

Status: NOT BUILT

Planned Nodes

Node 1 — Find Unconfirmed Bookings:

```
SELECT * FROM bookings
WHERE slot_datetime <= NOW() + INTERVAL '30 minutes'
    AND status = 'pending'
    AND reminder_2h_sent = TRUE
```

Node 2 — Release Slot:

```
UPDATE bookings
SET status = 'released', released_at = NOW()
WHERE booking_id = $1
```

Node 3 — Notify Waitlist (future):

If waitlist exists, offer released slot to next customer.

Node 4 — Flag for Follow-Up:

Log released booking for manual review.

Complete Node Chain (Current + Planned)

Create Booking:

Webhook → Calculate Deposit → Generate Token → Square Deposit → Insert DB → Send Confirmation

Confirm Handler:

Webhook → Validate Token → Update Status → Respond

T-24h Reminder 🟢:

Cron → Find Bookings Due → Send Reminder → Mark Sent

T-2h Reminder 📅:

Cron → Find Bookings Due → Send Urgent Reminder → Mark Sent

Auto-Release 🚧:

Cron → Find Unconfirmed → Release Slot → Notify Waitlist → Flag Follow-Up

Internal document — do not share with clients.

© 2026 Systack. All rights reserved.